NUMERICAL AND SIMULATION TECHNIQUES IN FINANCE
PROJECT II

Edward D. Weinberger, Ph.D., F.R.M
Adjunct Professor
Dept. of Finance and Risk Engineering
edw@wpq-inc.com/edw2026@nyu.edu

Compute the 1% VaR of a portfolio of 3 stocks, Microsoft (MSFT), Goldman Sachs (GS), and CitiGroup (C). Assume that returns for these stocks are jointly normal, the holding period is 1 day (1/252 year), and that you are given the following data:

Ticker	USDPrice Per Share	Ann Vol	Holding (Shares)
C	47	40%	100,000
GS	144	45%	-40,000
MSFT	31	30%	10,000

Correlation Matrix			
	C	GS	MSFT
C	1	0.6	0.2
GS	0.6	1	0.4
MSFT	0.2	0.4	1

The VaR can, in this case, be computed analytically, but the purpose of the assignment is to study the error introduced by doing Monte Carlo. Here's the analytical VaR calculation:

Let N_C, N_{GS}, N_{MSFT}, S_C, S_{GS}, S_{MSFT} be the respective numbers of shares and share prices of the three stocks in the portfolio. The value, V, of the portfolio at the beginning of the holding period is

$$V = N_C S_C + N_{GS} S_{GS} + N_{MSFT} S_{MSFT}$$

The change in value of the portfolio at the end of the holding period is

$$\delta V = N_C \delta S_C + N_{GS} \delta S_{GS} + N_{MSFT} \delta S_{MSFT}$$

If we really believe that the returns for these three stocks are normally distributed (which, in real life, is ridiculously simplistic, but never mind …), then we can drop the subscripts and write, generically,

$$\frac{\delta S}{S} = \sigma \sqrt{\delta t} Z,$$

or

$$\delta S = \sigma S \sqrt{\delta t} Z,$$

where σ is the volatility, δt is the length of the holding period, and Z is a normal random variable with mean zero and variance one (Since we will only care about the variance of V, we might as well assume that the returns of portfolio components, and thus the portfolio as a whole have mean zero.). We assume that S is known at the beginning of the period, so δS and thus δV also normally distributed with mean zero. If we knew the standard deviation, σ_V, of this latter random variable, we could compute its bottom 1st percentile as just $\alpha_{1\%}\sigma_V$, where $\alpha_{1\%}$ is the 1 percentile of Z, *i.e.*

$$0.01 = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\alpha_{1\%}} e^{-x^2/2} dx,$$

or about -2.3263.

But, under the simplistic assumption of normal returns, we have

$$
\begin{aligned}
Var\left[\delta V\right] &= E\left[\delta V^2\right] \\
&= E\left[\left(N_C \delta S_C + N_{GS} \delta S_{GS} + N_{MSFT} \delta S_{MSFT}\right)^2\right] \\
&= E\left[\sum_{\substack{\text{Ticker Pairs} \\ (T_1, T_2)}} N_{T_1} S_{T_1} N_{T_2} S_{T_2} \sigma_{T_1} \sigma_{T_2} Z_{T_1} Z_{T_2} \delta t\right] \\
&= \sum_{\substack{\text{Ticker Pairs} \\ (T_1, T_2)}} N_{T_1} S_{T_1} N_{T_2} S_{T_2} \sigma_{T_1} \sigma_{T_2} E\left[Z_{T_1} Z_{T_2}\right] \\
&= \sum_{\substack{\text{Ticker Pairs} \\ (T_1, T_2)}} N_{T_1} S_{T_1} N_{T_2} S_{T_2} \sigma_{T_1} \sigma_{T_2} \rho_{12} \delta t
\end{aligned}
$$

Note that, in the above, we have used the fact that

$$E\left[Z_{T_1} Z_{T_2}\right] = \rho_{12},$$

i.e. the entry in the correlation matrix with a row corresponding to Ticker_1 and a column corresponding to Ticker_2. Note also that those terms in the above sum in which $T_1 = T_2$ will have the correct form if $\rho_{11} = \rho_{22} = \rho_{33} = 1$.

In any case, we conclude that $\delta V = \alpha_{1\%} \sigma_V$, where σ_V is the square root of the above sum.

Now for the hard(er) part, *i.e.* the Monte Carlo. Here, we proceed directly from the formula for δV, computing it for many, many scenarios. Computing each of these scenarios requires a computation of the stock price changes, δS_C, δS_{GS}, and δS_{MSFT}. We have assumed that each of these changes is random according to the formula given above, so each scenario requires a different sample from the joint (normal) distribution of the stock price changes, where each sample value of δS_C is computed according to the formula

$$\delta S_C = \sigma_C S_C \sqrt{\delta t} Z_C,$$

each sample value of δS_{GS} is computed according to the formula

$$\delta S_{GS} = \sigma_{GS} S_{GS} \sqrt{\delta t} Z_{GS},$$

and each sample value of δS_{MSFT} is computed according to the formula

$$\delta S_{\text{MSFT}} = \sigma_{\text{MSFT}} S_{\text{MSFT}} \sqrt{\delta t} Z_{\text{MSFT}},$$

with the normal random variables Z_C, Z_{GS}, and Z_{MSFT} correlated according to the correlation matrix given above.

How do we compute correlated normal random variables? That's where the Cholesky algorithm discussed in class comes in. Recall that, for a given correlation matrix, $\mathbf{C}$, this algorithm computes a "square root" matrix, $\mathbf{R}$, such that $\mathbf{R}\,\mathbf{R}^T = \mathbf{C}$. It is then easy to check that the matrix-vector product $\boldsymbol{Y} = \mathbf{R}\,\boldsymbol{X}$, where $\boldsymbol{X}$ is a vector of independent normal random variables with unit variance, has correlation matrix $\mathbf{C}$

So, the assignment is to estimate the VaR via Monte Carlo from 1, 5, 10, 50, and 100 thousand scenarios. Run each estimate 20 times and compute the mean square error, relative to the analytic result. Do a log-log plot. Does it agree with the formula?

Notes:

It may be tempting to estimate the VaR as $\alpha_{1\%} s_V$, where s_V is an empirical estimate of σ_V. As discussed in class, *any* estimate based on a Monte Carlo calculation is a random variable, and thus subject to statistical fluctuation. Is this a better way of computing the 1% boundary than sorting the scenarios? As it happens, one can show that the error introduced by sorting the scenarios is typically about a factor of two *larger* than the error introduced by using s_V if we assume that these samples are normally distributed. However, the success of the s_V method is obviously highly dependent on the samples being truly normally distributed in the wings of the distribution, so please don't use it. Instead, sort the array and return the $\alpha_{1\%} N^{\text{th}}$ smallest value, as this method will work regardless of the distribution of the samples.

Also, please implement the Cholesky algorithm in VBA, if only because an EXCEL implementation is far less flexible (and far slower) than a VBA implementation.

In the implementation of numerical methods, it is critical to know what part of the code is responsible for most of the computational effort. In the present case, the most time-consuming part of the code will be the sort. The obvious methods of sorting typically require $O(N^2)$ comparisons, which will be time consuming for the values of N requested above. Fortunately, the so-called "quick sort" algorithm discussed in class usually requires $O(N \log N)$ comparisons (Why doesn't the base of the logarithm matter here?). Because this is a class in numerical methods, and not computer science, feel free to search the web – or ask your friends studying computer science – for a VBA implementation of quick sort. Keep in mind, though, that you are still responsible for *getting the quick sort code to work*. At a minimum, therefore, you should verify that an input list of random numbers is returned in sorted order.